

Case File Management & Analytics Portal (Legal Services)

Team : 11

P.Ruhitha	2520080053
M.Ananya	2520080069
S.Himasree	2520090181

Problem Statement

Legal case information is often maintained across multiple files, documents, and manual records, making it difficult to efficiently store, retrieve, update, and track case details. This can result in data duplication, inconsistent records, difficulty in monitoring hearings and case status, and delays in accessing important information. There is a need for a centralized database system that organizes legal case records and enables efficient retrieval, tracking, and analysis of case-related information.

Abstract

Legal case management involves handling a large amount of information, such as case details, clients, lawyers, judges, hearings, documents, and judgments. When these records are managed manually or stored across different files, it can become difficult to find information, track case progress, monitor hearings, and understand the workload of legal professionals. The proposed Case File Management & Analytics Portal is a centralized web application designed to make legal case management simpler and more organized. The system will store case-related information in a structured MySQL database, allowing users to easily add, update, search, and retrieve records. By connecting cases with their clients, lawyers, judges, hearings, documents, and judgments, the system provides a clear view of the complete history and current status of each case.

Abstract

The portal will also include SQL-based analytics to provide useful insights, such as pending and resolved cases, case distribution by type and status, active cases handled by lawyers, hearing statistics, document counts, and average case duration. These results will be presented through summary tables and dashboard metrics to make the information easier to understand. Overall, the project aims to improve the organization, consistency, accessibility, and retrieval of legal information while demonstrating key Database Management System concepts such as relational schema design, normalization, primary and foreign keys, constraints, joins, filtering, grouping, and aggregate queries. The system is developed using React and Vite for the frontend, FastAPI and Spring Boot microservices for the backend, and MySQL for database management.

Objectives

- To develop a centralized system for storing and managing legal case information.
- To reduce data duplication and inconsistencies caused by maintaining records across multiple files.
- To provide easy access to case details, clients, lawyers, judges, hearings, documents, and judgments.
- To enable efficient searching, updating, and retrieval of case information.
- To track case status, hearing schedules, and case progress.
- To generate useful analytics such as pending/resolved cases, case distribution, lawyer workload, and hearing statistics.
- To demonstrate DBMS concepts such as normalization, primary keys, foreign keys, constraints, joins, grouping, and aggregate queries.

Proposed System

Main Features:

- Case Management – Add, update, search, and track legal cases.
- Client Management – Maintain client information and link clients with their cases.
- Lawyer Management – Store lawyer details and track their active cases.
- Judge Management – Maintain information about judges handling cases.
- Hearing Management – Record hearing dates, outcomes, and next hearing dates.
- Document Management – Maintain case-related document records.
- Judgment Management – Store judgment details and final decisions.
- Analytics Dashboard – Display statistics about case status, types, hearings, workload, and duration.

Methodology

Requirement Analysis

Identify the information that needs to be managed, including cases, clients, lawyers, judges, hearings, documents, and judgments.

Frontend Development

Develop the user interface using React, HTML, CSS, and JavaScript. The interface will allow users to add, update, search, and retrieve case information.

Backend Development

Use FastAPI and Spring Boot microservices to handle communication between the frontend and the MySQL database and process user requests.

SQL-Based Analytics

Create SQL queries using joins, filtering, grouping, and aggregate functions to generate information such as pending/resolved cases, lawyer workloads, hearing statistics, document counts, and average case duration.

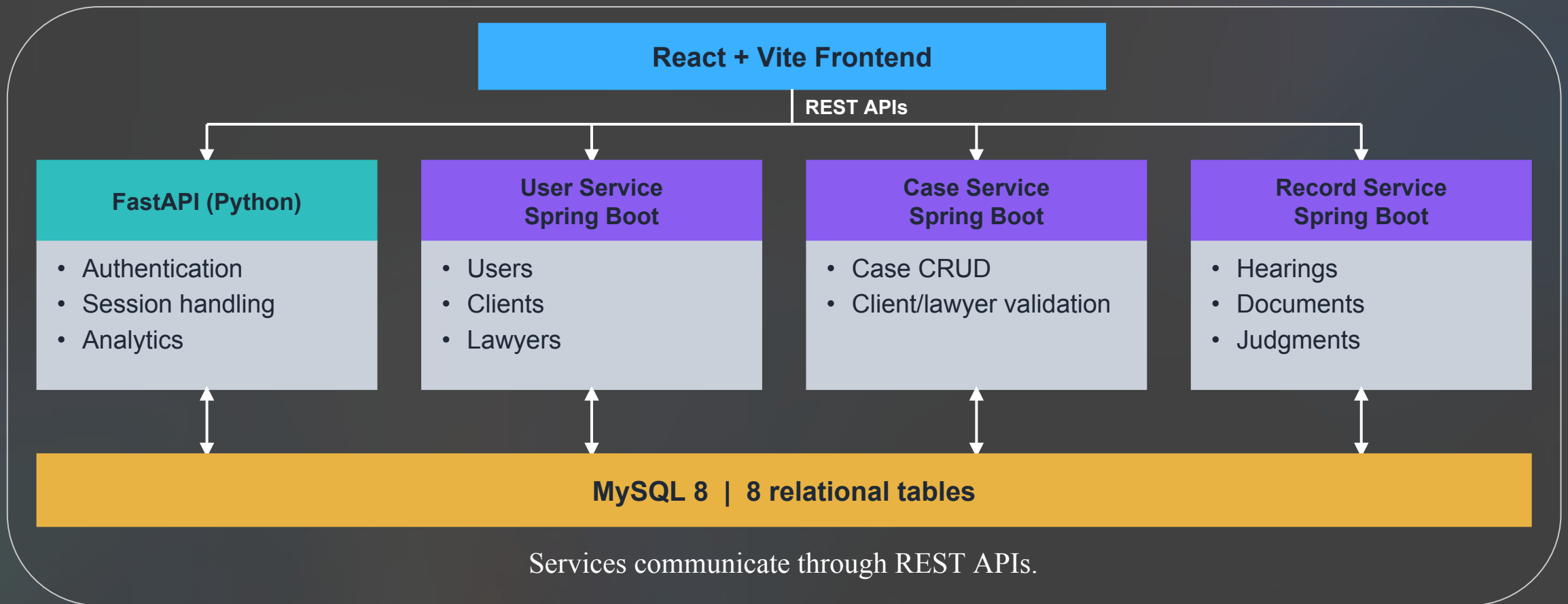
Dashboard & Testing

Present the analytical results through summary tables and dashboard metrics, and test the system to ensure that data can be correctly added, updated, retrieved, and analyzed.

Final Implementation Update

Component	Final implementation
Frontend	React + Vite, with Recharts for charts and dashboards
Backend	FastAPI (authentication, sessions, analytics) + Spring Boot microservices (User, Case, Record)
Communication	REST APIs between the frontend and all backend services
Database	MySQL 8 with 8 relational tables
Application roles	Lawyer and Client

System Architecture



Frontend Design

Role-Based Dashboards

Separate interfaces for Lawyer and Client users.

Hearing Tracking

Upcoming hearings, dates, courts and case timelines.

Easy Navigation

Sidebar navigation for cases, hearings, documents and analytics.

Data Visualization

Charts and dashboard cards present key case statistics.

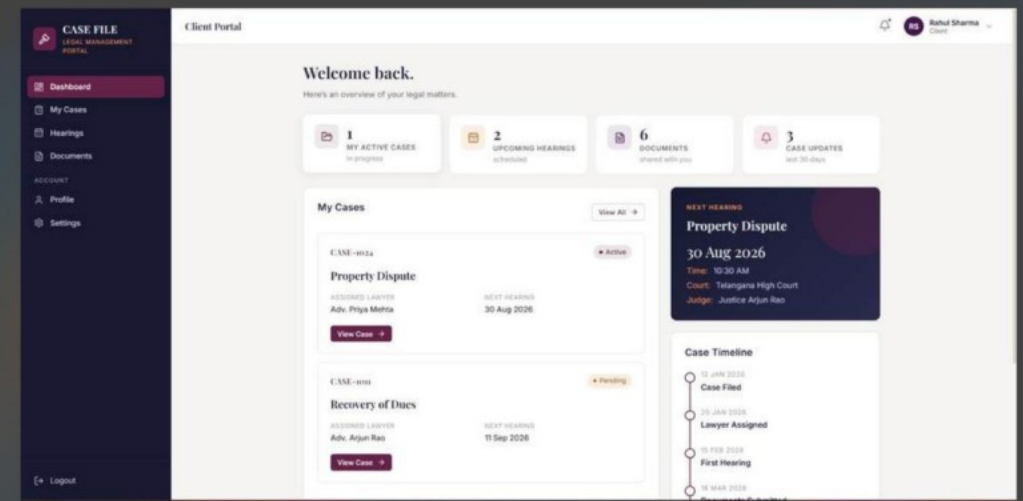
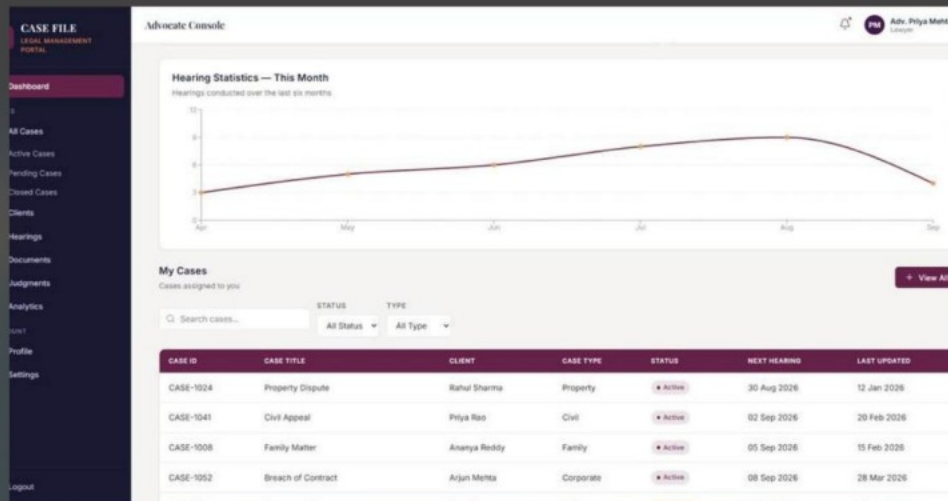
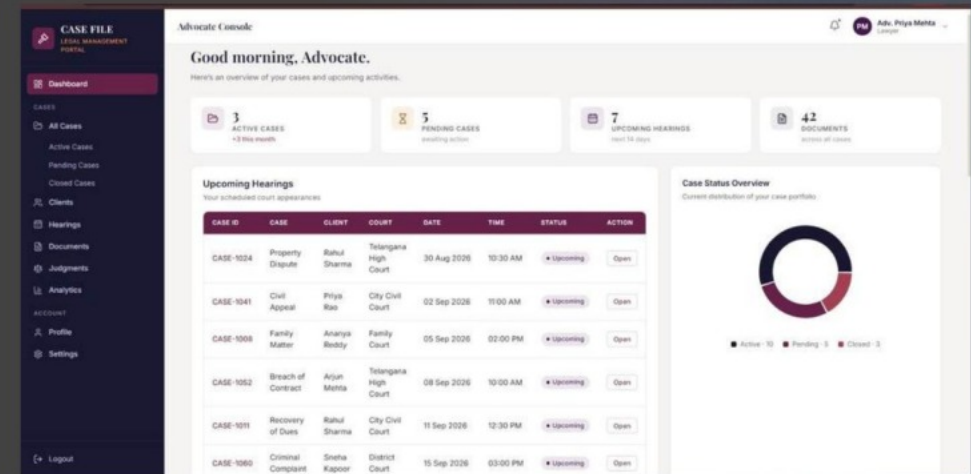
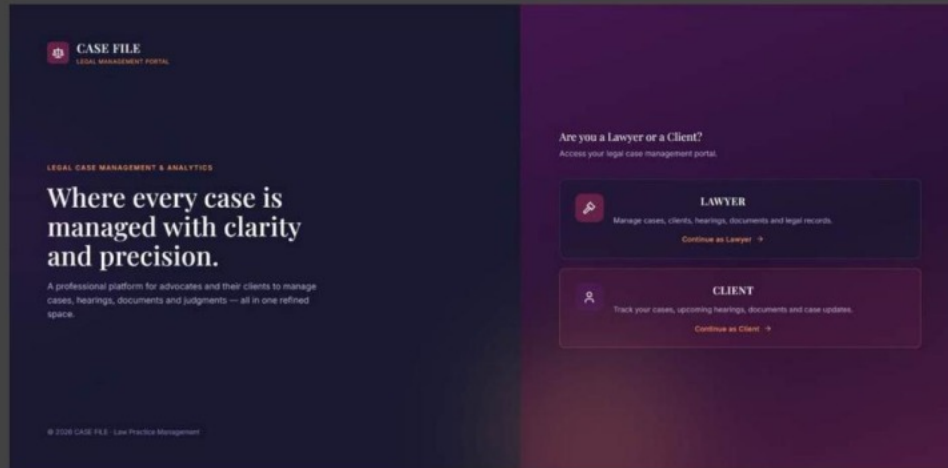
Case Management

Search, filter and track cases using clear status indicators.

User-friendly Interface

Buttons, forms, search bars and controls improve usability.

Frontend Implementation



Database Design

user

PK user_id • name • email
password • role • phone
UNIQUE: email

clients

PK client_id • FK user_id •
name • email password •
created_at • phone
UNIQUE: email

lawyers

PK lawyer_id • FK user_id
bar_number • specialization
experience_years • UNIQUE
bar no

judges

PK judge_id • name
court • specialization
contact details

cases - central table

PK case_id • UNIQUE case_number
title • case_type • description
filing_date • status • court
FK client_id → Clients
FK lawyer_id → Lawyers

Hearings

PK hearing_id
FK case_id • FK judge_id
hearing_date • hearing_time
court_room • purpose •
status

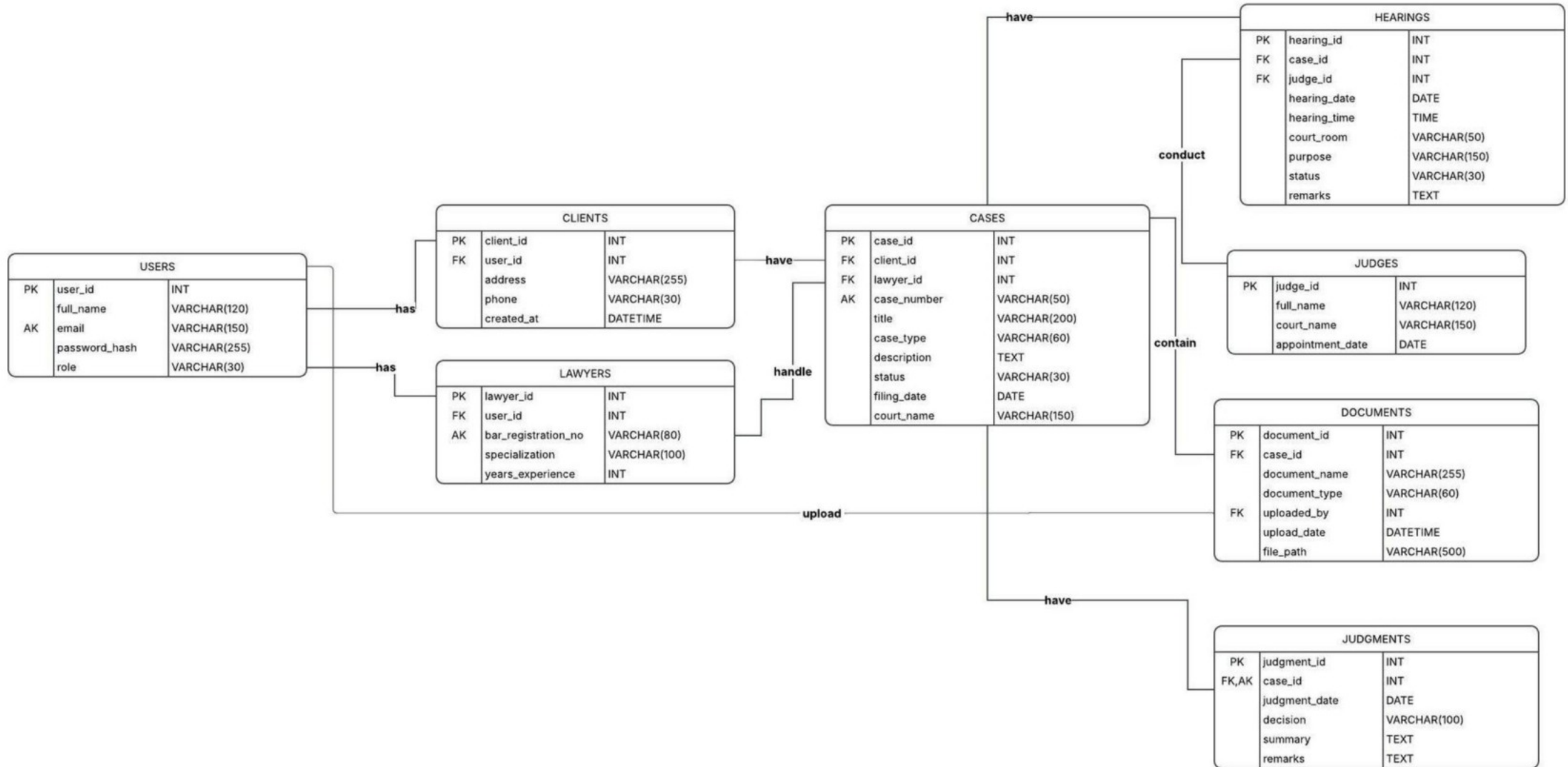
Documents

PK document_id
FK case_id • FK
uploaded_by
document_name • type
upload_date • file_path

judgements

PK judgment_id
FK + UNIQUE case_id
judgment_date • decision
summary • remarks
1 Case → max 1 Judgment

Database ER diagram



Backend & API Implementation

FastAPI (Python)

- Registration and login
- Authentication
- Session handling
- Analytics endpoint

User Service | Spring Boot

- Users
- Clients
- Lawyers

Case Service | Spring Boot

- Case CRUD
- Client/lawyer validation

Record Service | Spring Boot

- Hearings
- Documents
- Judgments
- Case validation

REST APIs connect the frontend and backend services, while MySQL provides persistent storage.

SQL Analytics & DBMS Concepts

Analytics Implemented

- Cases by Status
- Cases by Type
- Cases by Month
- Hearing Statistics
- Document Counts
- Judgment Counts

DBMS Concepts Demonstrated

- Relational schema design
- Normalization
- Primary Keys and Foreign Keys
- UNIQUE constraints
- JOIN operations
- WHERE / filtering
- GROUP BY and aggregate functions
- Referential integrity

Analytics are generated from the current MySQL database state.

1. Software Used

- React + Vite
- FastAPI (Python)
- Spring Boot (Java) Microservices
- MySQL 8
- REST APIs
- Git/GitHub

Testing & Results

API Testing

- Health endpoints
- CRUD endpoints
- REST communication

Database Testing

- Foreign-key validation
- UNIQUE case-number validation
- Referential integrity

Integration Testing

- Frontend and backend
- Microservice communication
- Services and MySQL

Results

- CRUD operations verified
- Duplicate case numbers rejected
- Database constraints validated
- Analytics updated according to database changes
- Frontend production build completed successfully

Challenges & Issues Faced

- Designing relationships between multiple legal entities.
- Avoiding duplicate data while storing information about cases, clients, lawyers, and hearings.
- Maintaining consistency between related tables using foreign keys.
- Handling different case statuses such as Active, Pending, and Resolved.
- Writing SQL queries involving multiple tables and joins.
- Generating meaningful analytics using GROUP BY and aggregate functions.
- Ensuring that updating or deleting records does not break relationships.
- Connecting the React frontend with the FastAPI and Spring Boot backend services and the MySQL database.
- Managing and retrieving large amounts of case-related information efficiently.

Literature Survey

1. eCourts Services – Government of India

- Provides case status, cause lists, court orders and judgments.
- Cases can be searched using CNR, case number, party name, advocate name, FIR number, case type, etc.
- Provides a case history showing events and orders associated with a case.
- NJDG provides large-scale judicial statistics and analytical information.

2. Clio Manage

- Provides centralized case, client, document and calendar management.
- Supports case-status tracking, document management, task management and billing.
- Provides customizable reports and dashboards for law-firm performance.
- Focuses on complete law-firm practice management, rather than only case records.

Literature Survey

3. MyCase

- Provides case management, document management and legal calendaring.
- Supports automated workflows, task management and client communication.
- Provides case analytics and customizable reports for caseloads, case status and firm performance.
- Designed primarily as a comprehensive commercial practice-management platform.

4. Indian Legal-Tech Platforms

- Platforms such as Provakil provide case tracking, court information, litigation workflows and analytics for Indian legal professionals.
- Modern platforms increasingly combine case management with court-data integration, alerts and AI-based legal features

Research Gap & Our Approach

eCourts Services

- Court information
- Case status
- Orders and judgments
- Case history

Clio

- Case/client management
- Documents
- Calendar
- Reporting

MyCase

- Case management
- Documents
- Workflows
- Analytics

Research / Application Gap

- Existing solutions are generally large-scale judicial information systems or comprehensive commercial legal practice-management platforms.
- Our project is a focused academic implementation of a relational DBMS-based legal case-management system.
- It demonstrates explicit entity relationships, normalization, primary/foreign keys, constraints, REST-based services and SQL-driven analytics.

Our Approach

Case Management

+

MySQL Relational
Database

+

REST APIs

+

Distributed Backend

+

Analytics Dashboard

Current Limitations

- Sessions are currently stored in FastAPI memory.
- Physical document upload/download is not fully implemented.
- Microservices are currently started separately.
- No email verification or password reset.
- No notification system.
- Deployment is currently local rather than cloud-based.

Future Work

- Add user authentication and role-based access for administrators, lawyers, and other authorized users.
- Add automated hearing reminders and notifications.
- Allow secure uploading and downloading of case documents.
- Add advanced dashboard visualizations and reports.
- Implement advanced search and filtering.
- Develop a mobile-friendly version of the portal.

Conclusion

The Case File Management & Analytics Portal provides a centralized and structured solution for managing legal case information. By connecting cases with clients, lawyers, judges, hearings, documents, and judgments, the system makes legal records easier to store, retrieve, update, and track.

The use of MySQL provides a reliable relational database structure with primary keys, foreign keys, constraints, and relationships. SQL-based analytics can provide useful insights into case status, lawyer workload, hearings, document counts, and case duration.

Overall, the proposed system reduces the limitations of manual and scattered record management while demonstrating important Database Management System concepts such as relational schema design, normalization, joins, filtering, grouping, and aggregate queries.

References

1. eCourts Services, Government of India. <https://ecourts.gov.in/>
2. Clio Manage, Clio. <https://www.clio.com/>
3. MyCase, Themis Solutions Inc. <https://www.mycase.com/>
4. Provakil, legal-tech platform for Indian legal professionals. <https://provakil.com/>
5. B. G. S. Grepon, "Designing and Implementing e-Justice Systems: An Information Systems Approach to Regional Trial Court Case Docket Management in Northern Mindanao, Philippines," 2021. <https://doi.org/10.5281/zenodo.4459814>
6. C. Sansone and G. Sperli, "Legal Information Retrieval Systems: State-of-the-Art and Open Issues," Information Systems, vol. 106, art. 101967, 2022. <https://doi.org/10.1016/j.is.2021.101967>
7. J. Tao and M. Zhang, "Development of Key Technologies of Legal Case Management Information System Based on J2EE," in Proc. ICIKM, 2022, pp. 49-53. <https://doi.org/10.1109/ICIKM56948.2022.00017>
8. O. S. Pirmatov, "Case Management in the Electronic Civil Case System," Tashkent State University of Law.
9. B. Liu et al., "Query Generation and Buffer Mechanism: Towards a Better Conversational Agent for Legal Case Retrieval," Information Processing & Management, vol. 59, no. 5, art. 103051, 2022. <https://doi.org/10.1016/j.ipm.2022.103051>
10. P. Bhattacharya, K. Ghosh, A. Pal and S. Ghosh, "Legal Case Document Similarity: You Need Both Network and Text," Information Processing & Management, vol. 59, no. 6, art. 103069, 2022. <https://doi.org/10.1016/j.ipm.2022.103069>

THANK YOU